

SEMESTER 3 • COURSE: DATA STRUCTURES

Data Structures

Introduction to Data Structures & Arrays

What is a Data Structure?

Types & Operations

Arrays in Depth

What is a Data Structure?

A Data Structure is a way of organizing, storing, and managing data so it can be used efficiently.

Think of it as a container that decides HOW your data is arranged.

1

Store

Keep data safely in memory

2

Organize

Arrange data in a useful order

3

Access

Retrieve or update data quickly

A Real-Life Analogy



A Bookshelf

Books are arranged in a specific order so you can find, add, or remove a book quickly — that arrangement IS a data structure.



Phone Contacts

Your contact list stores names in alphabetical order so you can search for a friend in seconds — an organized data structure at work.

Why Do We Need Data Structures?

1

Efficiency

Solve problems using less time and less memory

2

Organization

Keep large amounts of data manageable and clear

3

Speed

Search, insert, and update data much faster

Types of Data Structures

LINEAR

Data elements are arranged one after another, in a single sequence.

Array

Stack

Queue

Linked List

NON-LINEAR

Data elements are connected in a hierarchical or networked way.

Tree

Graph

Common Operations on Data Structures

1

Traversal

Visit every element one by one

2

Insertion

Add a new element

3

Deletion

Remove an existing element

4

Searching

Find a specific element

5

Sorting

Arrange elements in order

Introduction to Data Structures

1

A Data Structure organizes and stores data for efficient use

2

We use them for efficiency, organization, and speed

3

Two main types: Linear (Array, Stack, Queue) and Non-Linear (Tree, Graph)

4

Five basic operations: Traversal, Insertion, Deletion, Searching, Sorting

02

TOPIC 2

Arrays

What they are, how they work, and how to use them

What is an Array?

An Array is a collection of elements of the same data type, stored in contiguous (side-by-side) memory locations.

Example: an array of 5 exam scores



↑ *Index (position) of each element — arrays start counting from 0*

Key Characteristics of Arrays

1

Fixed Size

Size is set when the array is created and cannot change

2

Same Data Type

All elements must be of the same type (e.g., all integers)

3

Contiguous Memory

Elements are stored right next to each other in memory

4

Index-Based Access

Every element has a position number, starting from 0

Declaring an Array in Memory

```
int scores[5] = {72, 85, 90, 68, 77};
```

1

`int`

The data type — every element will be an integer

2

`scores`

The name we give the array

3

`[5]`

The size — this array holds exactly 5 elements

Types of Arrays

ONE-DIMENSIONAL (1D) ARRAY — a single row of elements



`arr[0]` `arr[1]` `arr[2]` `arr[3]` `arr[4]`

TWO-DIMENSIONAL (2D) ARRAY — rows and columns (like a table/grid)



Access using two indices: row and column — e.g., `grid[1][2]`

Array Operations: Traversal & Access

Traversal = visiting every element. Access = jumping straight to one using its index.



Direct access example: `scores[2]` instantly gives us **90** — no need to check the other elements first.

Array Operations: Insertion & Deletion

Inserting or deleting in the middle means shifting elements to make or close a gap.

BEFORE:



INSERT 25 at index 1 → shift 20,30,40 right by one:

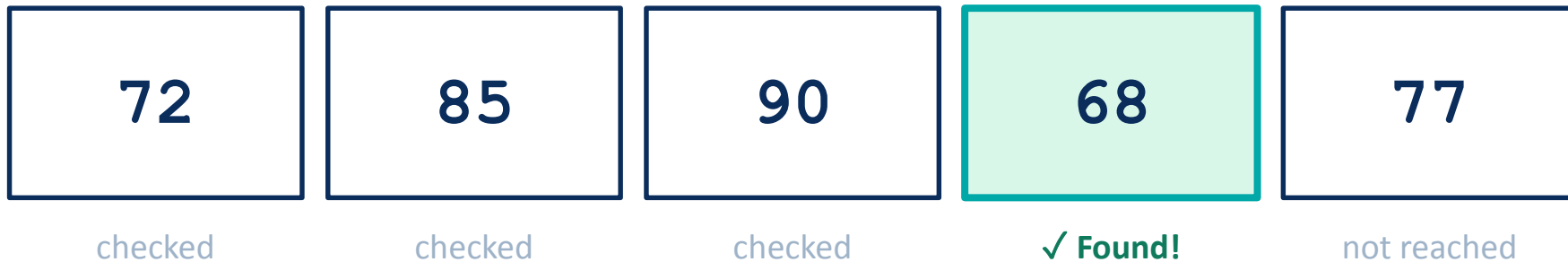


Deletion works the other way — remove an element, then shift the rest left to close the gap.

Array Operations: Searching

Linear Search: check each element one by one, from the start, until you find the value you want.

Example: Search for 68



Steps taken: 4 comparisons before finding 68 at index 3

Advantages & Disadvantages of Arrays

ADVANTAGES

- ✓ Fast access using an index
- ✓ Simple and easy to use
- ✓ Memory used efficiently (no extra pointers)

DISADVANTAGES

- ✗ Fixed size — cannot grow or shrink
- ✗ Insertion/deletion in the middle is slow
- ✗ Wastes memory if not fully used

QUICK REVISION

Exam-Ready Summary: Arrays

Definition

A collection of same-type elements in contiguous memory

Key Features

Fixed size, same data type, index-based access (starts at 0)

Operations

Traversal, access, insertion, deletion, searching

Trade-off

Fast access, but fixed size and slow middle insert/delete

Tip: draw the array boxes with indices on the board — it makes every concept click.